

Ah, I see exactly where the wires are getting crossed! What you are describing—typing / and instantly getting a massive autocomplete menu of commands, or seeing a visual side panel for your skills—is a phenomenal workflow.

But you are describing the **Claude Code CLI and its IDE extensions**, not the standard claude.ai consumer website.

Here is how those features actually map out, and why that / menu is exactly why you need the local terminal for your project.

The / Menu vs. The Web App

If you go to claude.ai in your browser and type / in the chat box, it doesn't give you a list of custom tools or model toggles.

When you type / and see "all kinds of shit" pop up—like /model, /clear, or custom tools—you are interacting with the **Claude Code command-line interface (CLI)**. If you are seeing this visually in a side panel, you are likely using an IDE (like VS Code with a Claude extension) or a cloud-based workspace that is running the CLI under the hood.

How Skills and Slash Commands Work Together

You are spot on about how the tools are built. Anthropic recently merged "Slash Commands" and "Skills" together in Claude Code.

Here is how that workflow actually operates behind the scenes:

1. **You build the tool:** You write a simple SKILL.md file (like the QAIRT model conversion script we talked about) and drop it in your .claude/skills/ folder.
2. **It instantly populates the menu:** Claude Code automatically reads that folder. The next time you type /, your custom tool is sitting right there in the autocomplete list alongside the built-in commands.
3. **The model sub-routing kicks in:** As we discussed earlier, if that SKILL.md file has model: haiku in its YAML block, selecting it from the / menu will instantly spawn a background agent to run your tool on a cheaper model without cluttering your main chat.

Why This Matters for Your Edge Project

The reason you cannot do this purely in the claude.ai web app is because of **execution**.

When you select a custom /skill from that menu, Claude actually executes the code against your local machine. If you created a skill called /compile-voice-model, Claude Code needs to reach into your Termux environment, grab the Snapdragon toolchain, and execute the Qualcomm conversion scripts locally.

The claude.ai web app is trapped in a cloud server. It can write the script for you, but it cannot run the /compile command because it cannot touch your local hardware.

If you love that / autocomplete workflow, you are in luck—that is exactly how the local CLI version of Claude Code is designed to be used.